BRAINFIELD SCHOOLS

Practical Build Guide

A practical blueprint for a Nigerian secondary school

Edition	Version 1.0 — 14 October 2026
Status	Proposed architecture and implementation plan. Application not built yet.
Core workflow	Subject teacher → Class teacher → VP Administration → Publication
Leadership	Separate director dashboard for results and statistics
Scope decision	Learning progress reports excluded

Prepared for the founders and school leadership. Review open policy decisions before implementation.

Contents

1. What to build first 3

2. Discovery checklist 3

3. Stack and version policy 3

4. Repository structure 4

5. Local setup when implementation starts 5

6. Database implementation order 5

7. Milestone A — Foundation and public website 6

8. Milestone B — Results and director dashboard 6

9. Milestone C — Timetables and learning resources 6

10. Milestone D — CBT and competition selection 7

11. Milestone E — Finance and payments 7

12. Milestone F — Programming classroom 7

13. Two-person collaboration and indicative schedule 8

14. Meaningful test plan 9

15. CI, staging, and production 9

16. Migration, training, and handover 9

17. Budget and selling the project 10

18. Regenerating Mermaid diagrams and PDFs 10

19. Definition of launch-ready 11

20. Official references 11

1. What to build first

Begin with the school website, accounts, academic structure, results workflow, and director dashboard. These establish the data and permissions needed by the remaining modules. Complete one real workflow from beginning to end before adding more dashboard screens.

The first useful release should allow an assigned subject teacher to enter scores, a class teacher to review the compiled class, the Vice Principal (Administration) to approve, an authorised publisher to release, and a student to view their report. The director should see statistics that reconcile with those published reports. Learning progress reports remain excluded.

Do not attempt all confirmed features in the first delivery. Optional parent access, attendance, admissions, library, clubs, and document requests need separate agreement.

2. Discovery checklist

Meet with the principal, VP Administration, a class teacher, subject teachers, director, and finance representative. Ask them to demonstrate their current process using anonymised examples.

Collect and approve:

- Class names/streams, student counts, subjects, terms, sessions, and teacher assignments.
- A blank report-card template, component maxima, grade bands, rounding rules, absence treatment, promotion rules, and tie policy.
- Who publishes results after VP approval and who reviews a class teacher's own subject scores.
- Whether directors may see pending results or only approved/published ones.
- Timetable periods, rooms, breaks, and teacher availability.
- CBT question types, expected simultaneous candidates, actual lab devices, power backup, and outage procedure.
- Fee items, due dates, instalments, discounts, scholarships, offline-payment process, and refund authority.
- Programming languages, teaching approach, and minimum supported device/browser.
- School-approved public content, photographs, awards, JAMB highlights, logo, and contact details.
- Data controller/operational responsibilities, retention, guardian access, support, and ownership expectations.

Produce a signed-off scope, role matrix, paper prototype, anonymised calculation examples, and pilot acceptance checklist. Record unanswered items in `DECISIONS.md`; do not turn guesses into school policy.

3. Stack and version policy

Use TypeScript, Node.js 24 LTS, Next.js App Router, React, Tailwind, Supabase PostgreSQL/Auth/Storage, Zod, React Hook Form, Paystack, CodeMirror, Vitest, and Playwright. Use Radix primitives and custom blue styling for accessible components. Use Recharts only when director charts are added.

At scaffolding, choose currently supported compatible releases, record exact resolved versions in the lockfile, and run security/compatibility checks. Avoid pinning an unverified future framework patch in this planning document. Node.js 24 is the proposed runtime line; verify current support before deployment. Use one package manager, npm, initially.

No Prisma is needed initially: use Supabase's generated database types, SQL migrations, and constrained PostgreSQL functions. Introduce another data layer only for a specific demonstrated problem.

4. Repository structure

The current repository contains documentation only. The following is the target structure after application scaffolding:

```
BrainField-Schools/
  README.md
  DESIGN.md
  CONTRIBUTING.md
  docs/
    SYSTEM-ARCHITECTURE.md
    BUILD-GUIDE.md
    DECISIONS.md
    diagrams/
      *.mmd
      rendered/
  apps/
    web/
      src/app/
        (public)/
        (auth)/
        (portal)/student/
        (portal)/teacher/
        (portal)/class-teacher/
        (portal)/vp-admin/
        (portal)/director/
        (portal)/admin/
        api/
      src/components/
      src/modules/
        school/
        results/
        reporting/
        assessment/
        classroom/
        competition/
        billing/
        programming/
      src/lib/
        auth/
        supabase/
        validation/
      tests/
  supabase/
    migrations/
    tests/
    seed.sql
  scripts/
  .github/workflows/
```

Route groups organise layouts; they do not provide security. Check authorisation inside server operations and database policies.

5. Local setup when implementation starts

Install Git, Node.js 24 LTS, an editor, and Docker if running the local Supabase stack. Create organisation-owned service accounts or clearly documented school-owned production accounts; developers use individual logins with delegated permissions.

From the repository root, run:

```
mkdir apps
npx create-next-app@latest apps/web --ts --tailwind --eslint --app --src-dir --use-npm --import-alias "
    @/*"
cd apps/web
npm install @supabase/supabase-js @supabase/ssr zod react-hook-form @hookform/resolvers @radix-ui/react
    -dialog @radix-ui/react-dropdown-menu
npm install --save-dev vitest @testing-library/react @testing-library/jest-dom jsdom
npm init playwright@latest
```

Install Recharts, CodeMirror, or language runtimes only in their relevant milestone. Inspect generated configuration rather than copying outdated Tailwind or Next.js conventions. Check dependencies before importing them. Commit the application lockfile.

In the repository root, set up local data tooling:

```
npm install --save-dev supabase
npx supabase init
npx supabase start
```

Docker must be running for the local stack. If Docker is unavailable, use a dedicated development Supabase project with synthetic data. Keep staging and production separate. Never use production student records for development.

Create `apps/web/.env.local` locally, not in Git. Add a sanitised `.env.example` when the code actually needs these settings:

```
NEXT_PUBLIC_SUPABASE_URL=development-project-url
NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY=development-publishable-key
SUPABASE_SECRET_KEY=server-only-secret-for-controlled-admin-jobs
PAYSTACK_SECRET_KEY=server-only-test-key
APP_ORIGIN=http://localhost:3000
RUNNER_ORIGIN=separate-programming-runner-origin
CRON_SECRET=server-only-random-worker-secret
```

Use the key names required by the chosen current Supabase SDK/project configuration. The publishable key identifies the project; RLS and grants protect exposed records. Secret/service-role keys bypass RLS and must remain server-only. `PAYSTACK_SECRET_KEY` starts in test mode. A `NEXT_PUBLIC_` variable is visible in browser bundles; never apply that prefix to secrets.

Run the generated app from `apps/web` with `npm run dev`. Before shared work begins, add scripts for linting, `tsc --noEmit`, unit/integration tests, and browser tests. These application scripts do not exist in the planning repository yet.

6. Database implementation order

1. School, profiles, memberships, explicit role grants, and audit events.
2. Sessions, terms, classes, student enrolments, subjects, registrations, teacher assignments, and class-teacher assignments.
3. Versioned assessment/grading policies and permission helper functions.

4. Subject sheets, scores, result batches, batch-sheet revisions, approvals, and published reports.
5. Transactional outbox, jobs, and notification delivery records.
6. Classroom resources/timetables, then exams/attempts, then finance, then programming submissions.

Write forward migrations and realistic synthetic seeds. Put constraints beside the tables they protect. Enable RLS before exposing a table and restrict grants. Add indexes for actual query patterns. SQL reporting views must preserve caller permissions; do not accidentally bypass RLS through definer privileges.

Use database functions for approval/publication/payment transactions. Validate caller membership and assignment inside the function. Keep `SECURITY DEFINER` functions minimal, with fixed search paths and restricted execute grants. Test both authorised and denied operations using real user contexts.

Generate TypeScript database types after migrations. Review a database diff before deployment. Never modify an already deployed migration to hide a schema change.

7. Milestone A — Foundation and public website

Build approved public pages and content publishing first. Set up authentication, role-based navigation, class/term configuration, and staff invitations. Provide usable empty states; do not fill dashboards with made-up live metrics.

Account onboarding should accommodate students without email addresses through a school-approved account-recovery model. Decide this before using admission numbers as login identifiers. Do not create predictable passwords. Staff with privileged roles use MFA.

Acceptance: a public visitor cannot access portal records; each pilot role sees the correct workspace; another class's record IDs remain inaccessible; deactivated staff lose access; public content has an identified approver.

8. Milestone B — Results and director dashboard

Implement a vertical slice with one synthetic class, two subjects, and separate teacher/class-teacher/VP/publisher/director/student accounts.

Build score entry and CSV import with preview, row-level errors, maxima checks, unknown-student detection, duplicate detection, and an explicit commit step. Match students through stable admission identifiers, never names alone. Treat spreadsheet cells as data and sanitise exported cells to prevent spreadsheet formula execution.

Add sheet submission, class compilation, return-with-comments, class approval, VP approval, and publication. Use revision checks and transactions. Freeze report-card snapshots with grading-policy version, source references, publisher, and publication date. Generate printable HTML first; add downloadable PDF reports through a bounded worker if needed.

Build the director's filters and metric definitions from the architecture. Reconcile each aggregate with a detailed export. Keep pending approval counts separate from published performance statistics. Do not include learning progress reports.

Acceptance: all workflow states and correction paths work; no reviewer can silently change a teacher's marks; concurrent changes cause an understandable conflict; unpublished records stay hidden from students; printed totals match approved examples; director statistics match published records.

9. Milestone C — Timetables and learning resources

Add class timetables with teacher/class/room conflict detection. Enforce overlap checks safely under concurrent requests. Build student-owned study plans with editable sessions and simple scheduling rules. No AI service is required.

Add assignments with versioned submissions, due dates, permitted file types, and teacher feedback. Add ICT e-notes with sanitised rich text, draft/publish states, and class/term/topic filters. Lazy-load heavy editors. Make private files downloadable only after authorisation.

Acceptance: teacher assignments constrain access; deadlines use school time correctly; students can edit only their own study plans/submissions; unsafe uploads and cross-class downloads are rejected; the interface works on the target phones.

10. Milestone D — CBT and competition selection

Start with practice multiple-choice tests. Then add scheduled formal exams, versioned question banks, exam roster, random ordering persisted per attempt, server deadlines, autosave, resume, submit, expiry finalisation, and controlled score release. Essays/manual grading come after the basic attempt lifecycle works.

Test delayed and repeated save requests, two browser tabs, refreshing, device clock changes, disconnected submission, and expired attempts. Do not expose answer keys in HTML, API responses, browser storage, or logs. Formal exam outages follow a school-approved exception process.

Next add versioned competition cutoffs and eligibility based on published results. Freeze qualification inputs, run selection tests, apply agreed tie rules, and allow audited final review. Practice remains available according to school policy.

Acceptance: attempt state survives refresh; stale saves cannot overwrite newer answers; timeout is enforced on the server; hidden answers stay private; repeat finalisation is harmless; qualification is reproducible from a rule version and source revision.

11. Milestone E — Finance and payments

Build invoice/item generation and balances before checkout. Store money in integer kobo, with NGN currency. Define instalment, scholarship, discount, overpayment, and refund policies with the finance representative.

Use the school's Paystack test account first. The server creates the payment attempt with authoritative amount/reference. Use hosted checkout. Validate webhook signatures over raw request bytes; durably record and deduplicate events; verify transaction status and expected details server-side. Apply allocations transactionally and issue one receipt per confirmed payment.

Build a reconciliation screen for pending/mismatched payments, failed events, reversals, and settlement differences. Manual/offline payment confirmation needs a separate permission and supporting evidence, never a student-editable paid checkbox.

Acceptance: forged callbacks fail; signature failures cannot mutate balances; repeated events credit once; wrong currency/amount/reference is held for review; refunds/reversals preserve history; finance staff can reconcile test cases independently.

Activate live payments only after school acceptance and merchant-account ownership are confirmed. Provider fees, taxes, and settlement terms require current verification.

12. Milestone F — Programming classroom

Begin with lessons, CodeMirror exercises, source saving, and teacher review. Introduce HTML/CSS/JavaScript practice in a separate-origin sandbox with restricted messaging. For Python, evaluate Pyodide on actual lab hardware and network conditions before committing to it.

Enforce execution time/output limits, reset/stop controls, network restrictions, and safe rendering. A worker on the portal origin alone is not sufficient isolation. Browser-generated pass/fail results are practice feedback, not authoritative

exam grades.

If authoritative automatic grading is needed, scope a separate sandbox service with strict isolation, no production secrets, resource quotas, and an independent security review. It is not part of the initial web hosting process.

Acceptance: submitted programs cannot read portal credentials or private records; infinite loops can be stopped; output is capped and escaped; teacher review works without executing code on the app server.

13. Two-person collaboration and indicative schedule

Period	Collaborator A focus	Collaborator B focus	Joint completion gate
Weeks 1–2	School discovery, layouts, public content	Data model, role matrix, environment setup	Workflow and pilot scope approved
Weeks 3–4	Public website, onboarding, portal shell	RLS, identity, academic configuration	Role isolation demonstrated
Weeks 5–8	Score entry, review UI, report cards	Atomic result functions, reporting queries	Full results pilot and director metrics
Weeks 9–11	Timetables, assignments, ICT notes	File policies, CBT attempt model	Classroom workflows and practice CBT
Weeks 12–14	Formal exam UI, competition screens	Save/recovery/finalisation, qualification	CBT load and recovery acceptance
Weeks 15–17	Billing UI and receipts	Payment verification/reconciliation	Finance acceptance in test mode
Weeks 18–20	Programming exercises and runner UI	Isolation, jobs, monitoring, restore	Programming and operations acceptance
Weeks 21–22	Training and pilot feedback	Migration and deployment hardening	School launch decision

This is a planning range, not a promise. More experienced developers may overlap milestones; beginners, part-time work, weak infrastructure, or slow school feedback will extend it. Release the academic MVP earlier if it is ready. Swap ownership based on your strengths and review each other's work. Do not divide all frontend/backend work without agreeing on contracts and testing the integrated flow weekly.

14. Meaningful test plan

Test level	What matters
Unit	Grading, rounding, absence handling, timetable conflicts, money arithmetic, cutoff/tie rules
Database/RLS	Own versus another student's records; assigned versus unassigned teachers; denied role changes; cross-school references
Integration	Complete result transitions, source-revision invalidation, payment allocation, outbox atomicity
Browser	Subject entry through publication; director filtering; student report access; exam reconnect; payment status
Concurrency	Double publication, two payment events, delayed CBT saves, overlapping timetable edits
Accessibility	Keyboard, focus, labels, contrast, zoom, screen-reader spot checks, chart tables
Performance	Actual expected CBT concurrency, large score grids, low-end phones, throttled connections
Recovery	Restore database and file objects together; expired job leases; failed notifications; payment reconciliation

Use synthetic fixtures with expected outputs reviewed by the school. Repeating the implementation in a test is insufficient: assert observable academic and financial invariants. Include explicit denied-access cases, not only successful screenshots.

15. CI, staging, and production

For application pull requests, run dependency installation from the lockfile, lint, types, meaningful tests, production build, and SQL/RLS checks. Run browser tests against a synthetic staging environment. Scan for accidental secrets and vulnerable dependencies. Keep production credentials away from untrusted fork builds.

For documentation changes, validate Mermaid rendering, internal links, PDF text/pages, and consistency between Markdown/source diagrams. The initial repository includes a documentation check workflow; application checks are added when the application exists.

Provision separate Vercel/Supabase environments. Configure DNS, TLS, allowed auth redirect URLs, session/cookie settings, webhook URLs, cron authentication, and rate limits. Restrict preview indexing and access when appropriate. Do not place real student data in preview deployments.

Use reviewed additive migrations and compatible releases. Document rollback steps. If a database change is irreversible, explain the recovery approach before release. School staff should approve the pilot before production account invitations or real payments.

16. Migration, training, and handover

Create a structured import template for admission number, name, class, subject registration, and permitted guardian/contact fields. Validate duplicates, class mapping, required data, and consent/publication status where applicable. Rehearse imports using synthetic data, then run controlled private imports with row counts and checksums. Never commit import files.

Pilot one class and a small set of teachers. Compare report cards with the existing approved manual process. Train subject teachers, class teachers, VP Administration, publisher, director, and finance users separately. Give each role a

short task guide.

Handover should include account ownership, domain access, deployment process, backup/restore procedure, incident contacts, support hours, renewal responsibility, data export procedure, and administrator recovery. Confirm the two developers' future availability; avoid a system that only one person can maintain.

17. Budget and selling the project

Treat development, hosting, support, messaging, and payment fees as separate budget lines. Obtain current provider quotations before committing to a school price. Useful cost categories include domain renewals, database compute/storage/backups, web hosting, scheduled work, file egress, monitoring, SMS/email, support time, training, and lab/power/network upgrades.

Estimate each milestone as person-days multiplied by an agreed internal day rate, plus direct expenses, contingency, and support commitments. Quote optional modules separately. Re-estimate after discovery, especially if exclusive code ownership is requested. Keep your internal cost calculations and negotiating limits private; they do not belong in a public source repository.

Shared software prices provide affordability context, not direct custom-project valuation. As checked on 14 October 2026, SmartEdu advertises a Pro rate of ₦600 per active student per term, and LumiSaS lists plans from ₦500 to ₦2,000 per active student per term. Compare included modules, support, and service limits before drawing conclusions.

Use milestone payments tied to accepted deliverables; for example, discovery/start, academic pilot, and accepted handover. Define a bounded defect warranty and a separate maintenance agreement. Clarify whether infrastructure is billed directly to the school or included with stated limits. Decide software reuse/licensing before publishing implementation: this public planning repository currently grants no open-source licence.

18. Regenerating Mermaid diagrams and PDFs

The `.mmd` files are extracted from the Mermaid blocks in `SYSTEM-ARCHITECTURE.md` in document order. Edit the Markdown source, then regenerate. Do not maintain a different architecture only in a PDF.

Install the documentation tools at the repository root:

```
npm ci
python -m pip install -r requirements-docs.txt
```

Mermaid CLI uses a Chromium-based browser. Install a supported browser using Puppeteer's browser installer if needed, or set `PUPPETEER_EXECUTABLE_PATH` to an existing Chrome/Edge executable. Do not commit machine-specific browser paths. Then run:

```
python scripts/render_diagrams.py
python scripts/build_documents.py
python scripts/check_documents.py
```

`render_diagrams.py` produces actual Mermaid-rendered SVG and PNG files; PNGs are embedded in the PDF for compatibility. It reuses unchanged renders; set `MERMAID_FORCE_RENDER=1` in your shell when changing renderer configuration or dependency versions. The document checker validates local links, diagram-source agreement, non-empty diagram assets, and key PDF content. Visually inspect representative pages after layout changes. Commit verified source, renders, and PDFs together.

19. Definition of launch-ready

The product is ready for a controlled launch when the school has accepted the result workflow, role isolation, report accuracy, real-device experience, data-import checks, backup restoration, and support ownership. CBT and payments each have their own additional acceptance gate. A visually finished dashboard alone is not a completed school system.

20. Official references

- Framework setup: https://nextjs.org/docs/app/getting-started/installation
- Node.js lifecycle: https://nodejs.org/en/about/previous-releases
- Supabase server-side auth: https://supabase.com/docs/guides/auth/server-side/nextjs
- Database security: https://supabase.com/docs/guides/database/postgres/row-level-security
- Storage: https://supabase.com/docs/guides/storage/security/access-control
- Backup limitations: https://supabase.com/docs/guides/platform/backups
- Payment verification: https://paystack.com/docs/payments/verify-payments/
- Webhook signatures: https://paystack.com/docs/payments/webhooks/
- Pyodide worker use: https://pyodide.org/en/stable/usage/webworker.html
- Browser testing: https://playwright.dev/docs/intro
- Interface guidelines: https://vercel.com/design/guidelines
- Pricing comparison: https://smartedu.ng/pricing and https://lumisas.com/pricing

Check current official documentation again when implementation begins.